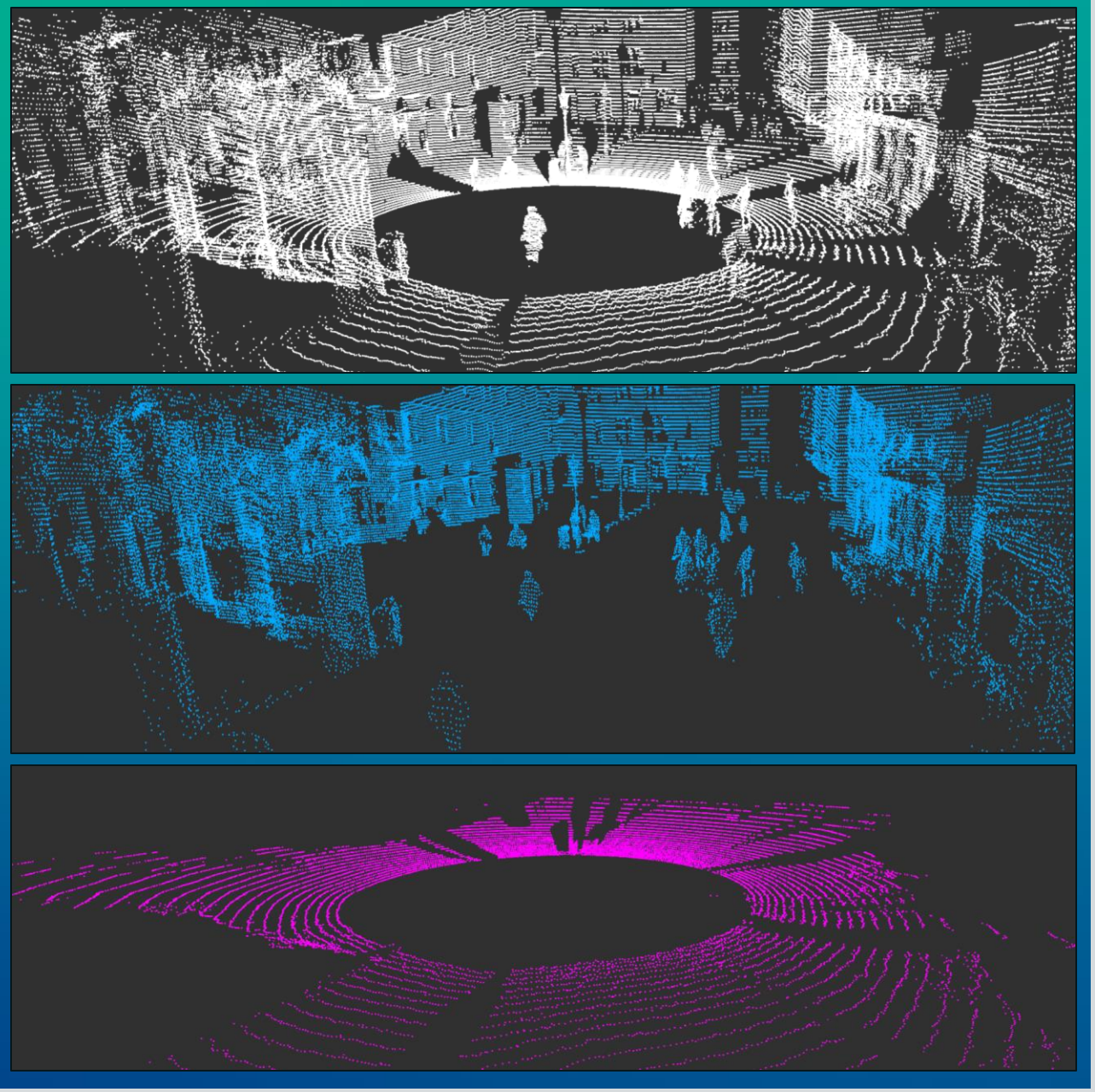


Data: Pfreundschuh et al., ICRA 2021 [1]

Deutsches Forschungszentrum für Künstliche Intelligenz
German Research Center for Artificial Intelligence

GSeg3D: A High-Precision Grid-Based Algorithm for Safety-Critical Ground Segmentation in LiDAR Point Clouds

Muhammad Haider Khan Lodhi, Christoph Hertzberg

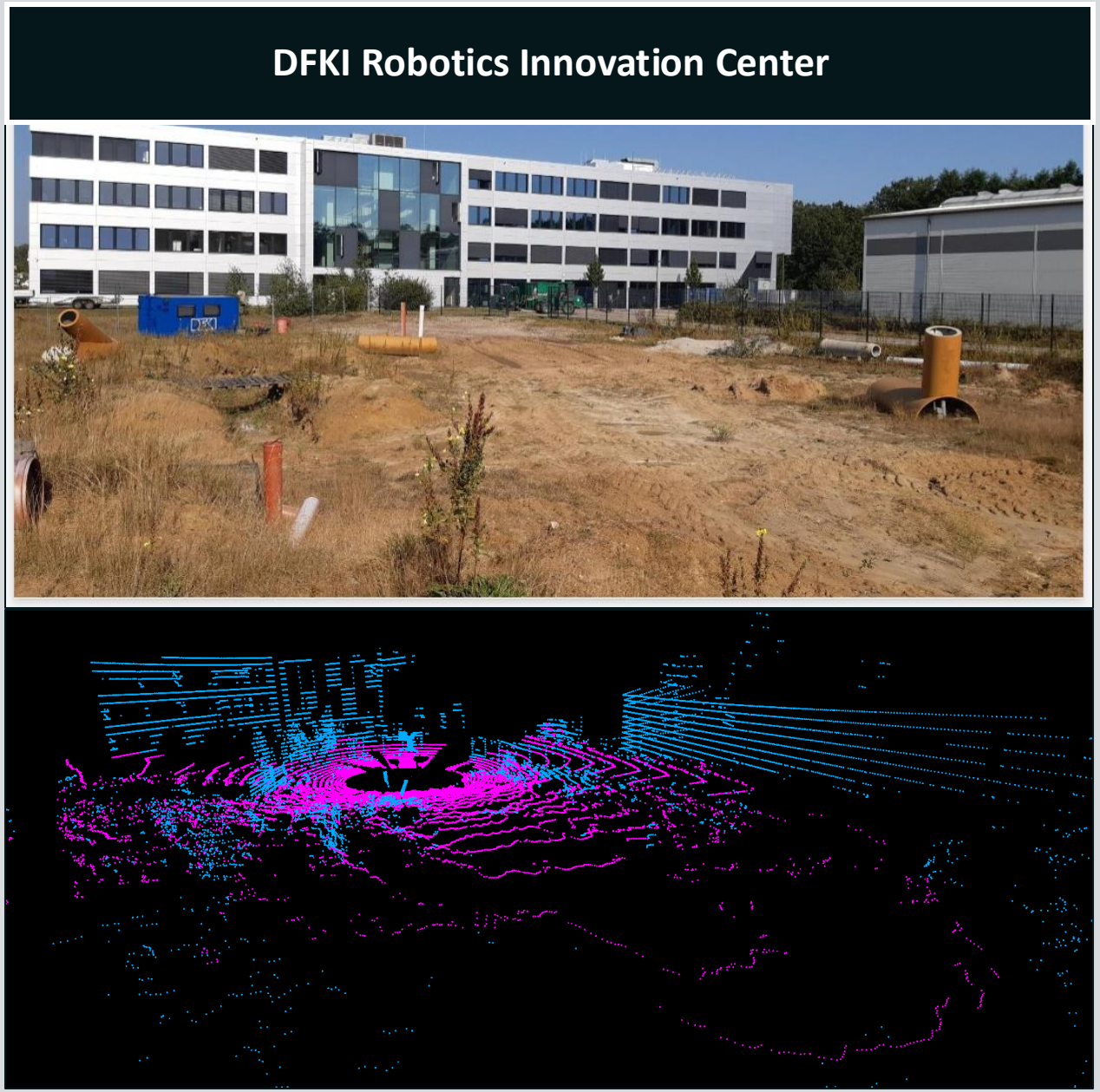
DFKI – German Research Center for Artificial Intelligence

ICRCV 2025, Hong Kong

Session 1: Mobile Robots and Intelligent System Design

Motivation and Context

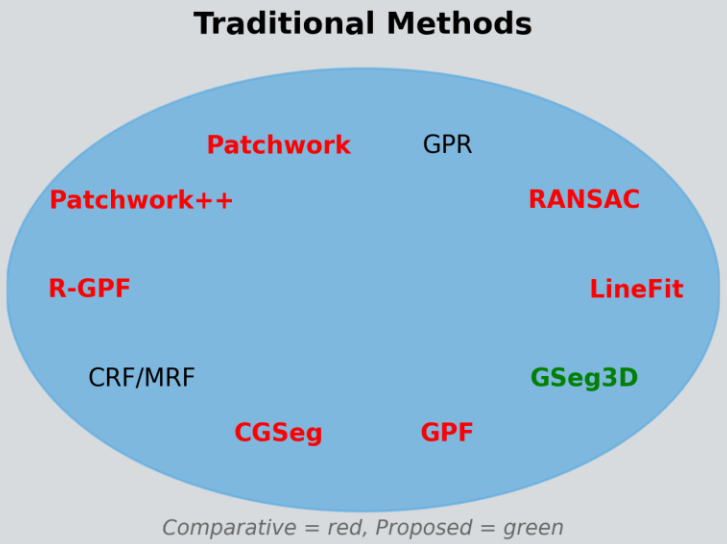
- Ground segmentation forms the basis of perception, traversability, and safety
- Mislabeling ground or obstacles leads to unsafe and unreliable decisions
- Demands **high precision**, **low variance**, and **real-time consistency**
- Relevant for both **autonomous vehicles** and **field robotics applications**



Ground Segmentation in Point Clouds

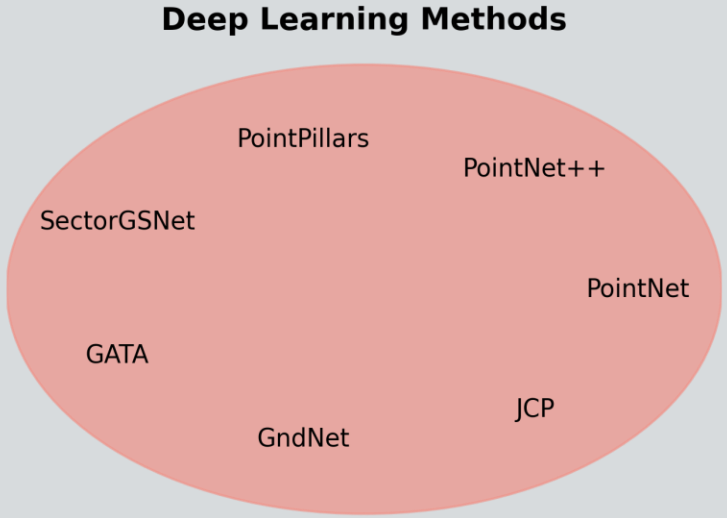
Traditional Methods (Model-Driven)
High precision on flat terrain but fragile in complex scenes

- **Geometric Fitting:** GPF, R-GPF, LineFit, RANSAC
- **Progressive Refinement:** Patchwork, Patchwork++, CGSeg
- **Probabilistic:** GPR, CRF/MRF



Deep Learning Methods (Data-Driven)
High performance but require significant data and compute

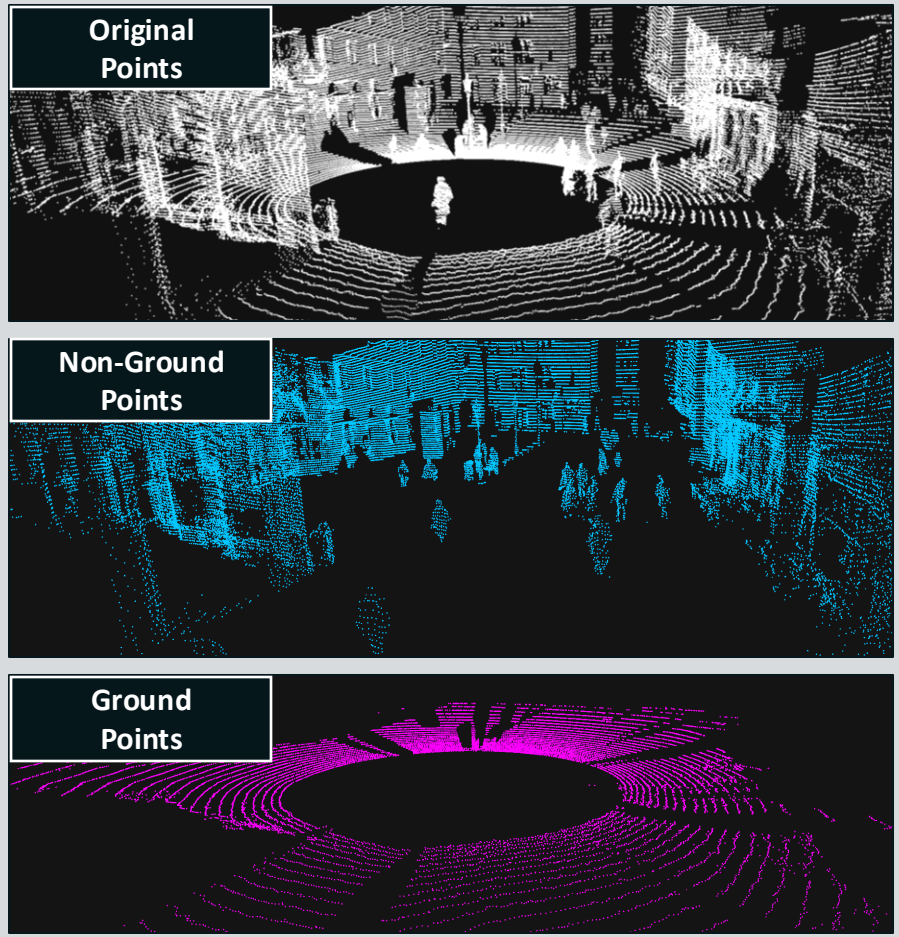
- **Grid-Based:** SectorGSNet, GndNet
- **Graph-Based:** GATA
- **Point-Based:** PointNet, PointNet++, JCP



GSeg3D Overview

GSeg3D integrates the strengths of multiple classical approaches:

- **Grid-Based:** Divides the point cloud into a regular 3D grid for initial organization
- **Progressive Refinement:** Employs a dual-phase segmentation (coarse-to-fine) for speed and precision
- **Region Growing:** Uses KD-Tree expansion to bridge scan-line gaps and grow regions
- **Geometry Fitting:** Applies plane model fitting to identify planar ground structures

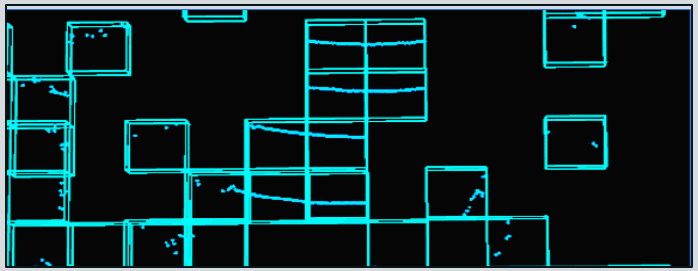


Key Outcome: Achieves **high precision with low standard deviation**, making it suitable for **safety-critical applications** in both structured and unstructured terrains.

Challenges in Grid-Based LiDAR Data Processing

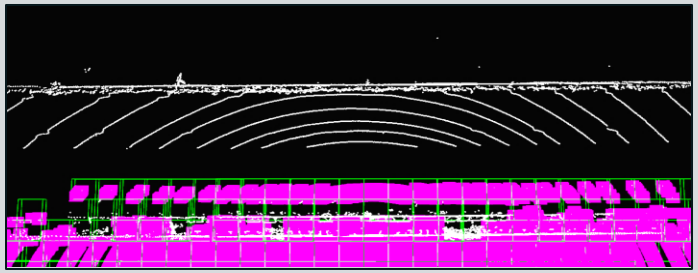
Data Inherent Challenges

- **Sparse & Irregular Data:** LiDAR point clouds are inherently sparse, leading to information gaps in a fixed grid.
- **Complex Environments:** Mixed structures, uneven terrain, and vegetation confuse fundamental algorithms like plane fitting.



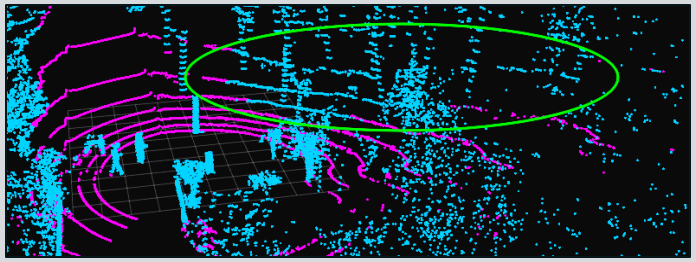
Algorithmic Limitations

- **Connectivity Loss:** Grid cells disrupt the natural connectivity of surfaces and linear features.
- **Transition Ambiguity:** Leads to over-segmentation (false ground) and under-segmentation (false non-ground) at ground-obstacle boundaries.



Systematic Tradeoffs

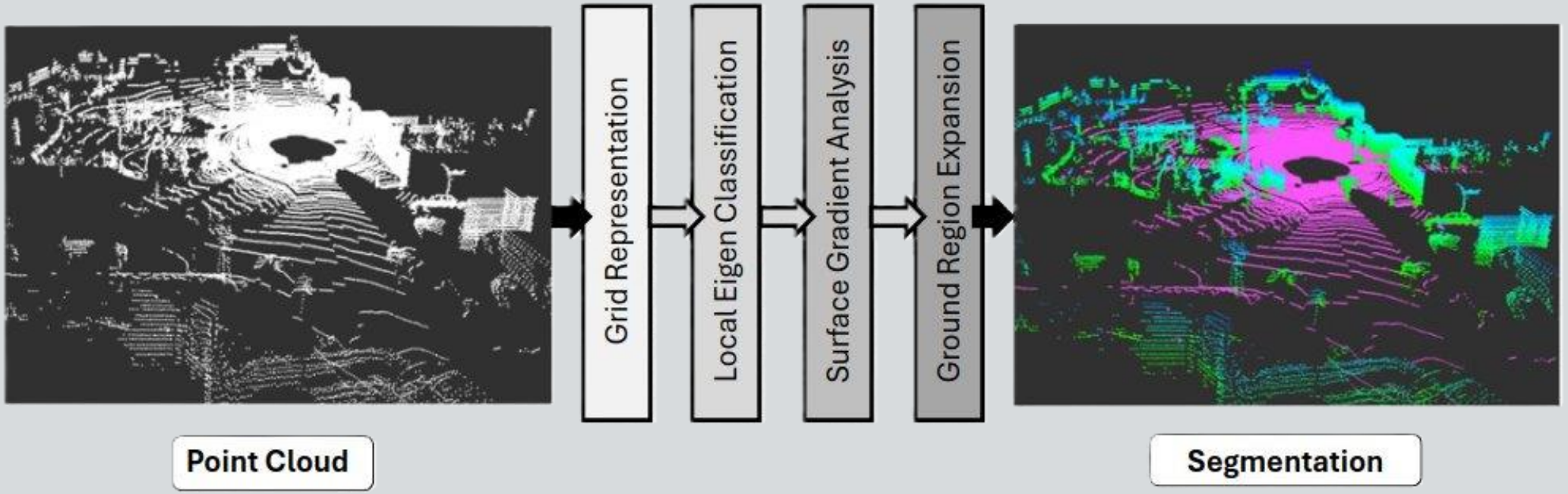
- **The Precision-Recall Dilemma:** Optimizing for accurate ground points detection (precision) often reduces the ability to find all ground points (recall), and vice versa.



Pipeline Overview

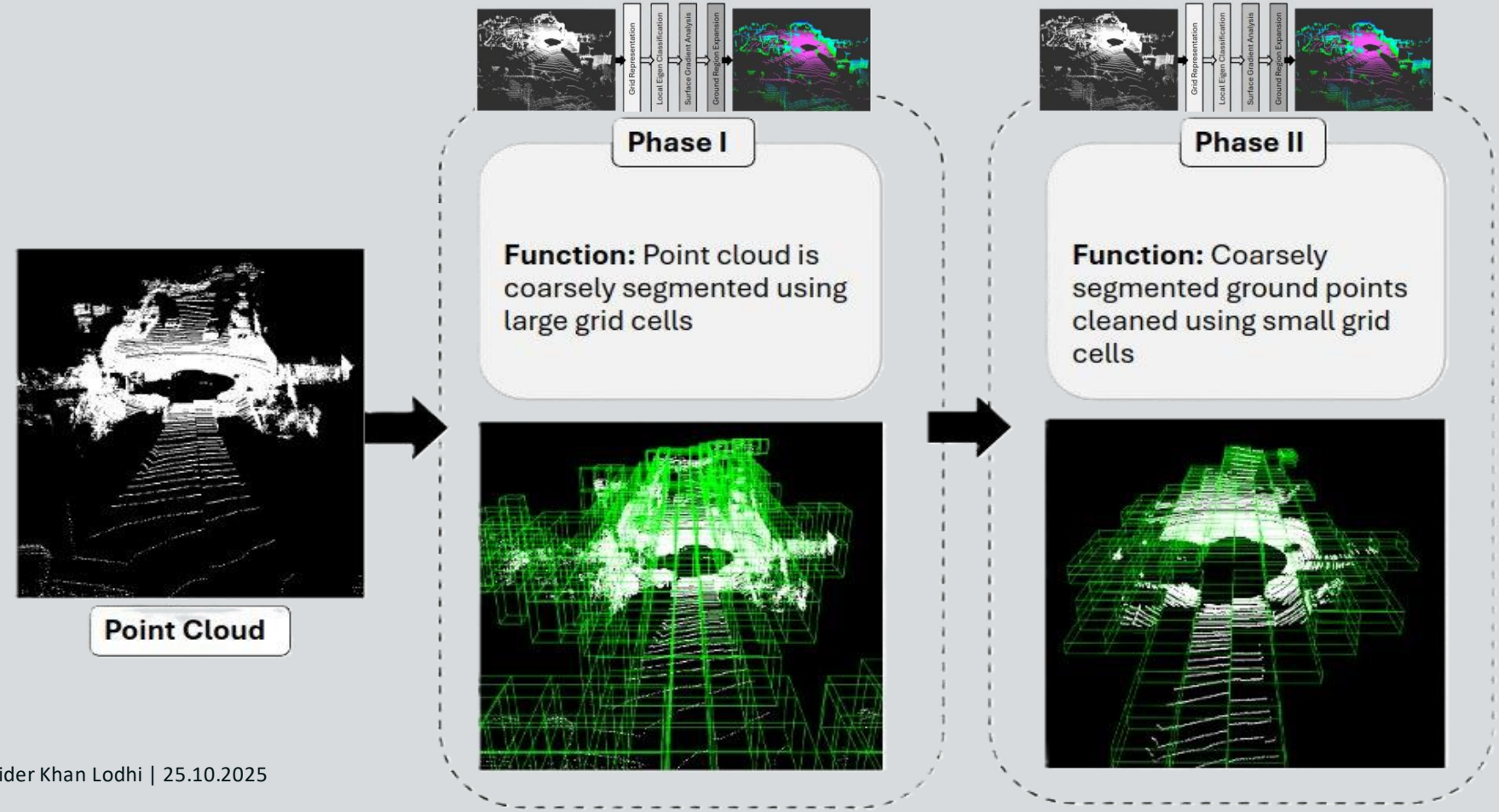
The segmentation is structured as a **four-stage pipeline**, designed to **adapt to diverse ground conditions and scenarios** with minimal parameter tuning

1. Grid Representation
2. Local Eigen Classification
3. Surface Gradient Analysis
4. Ground Region Growth



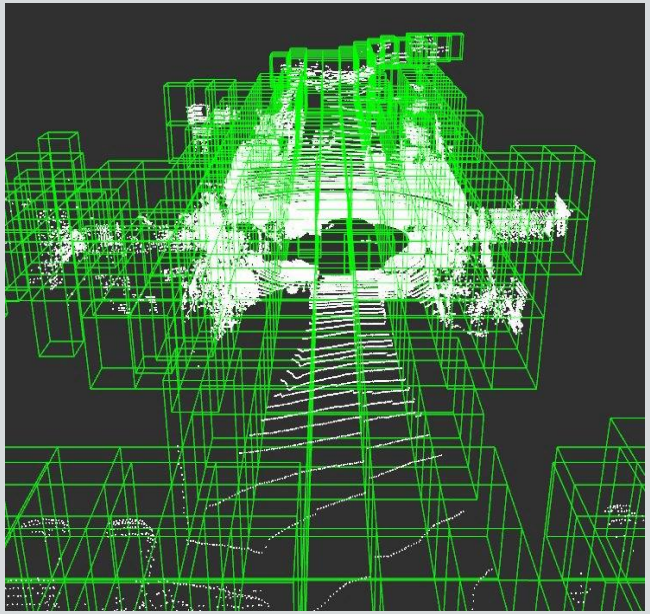
Pipeline Overview

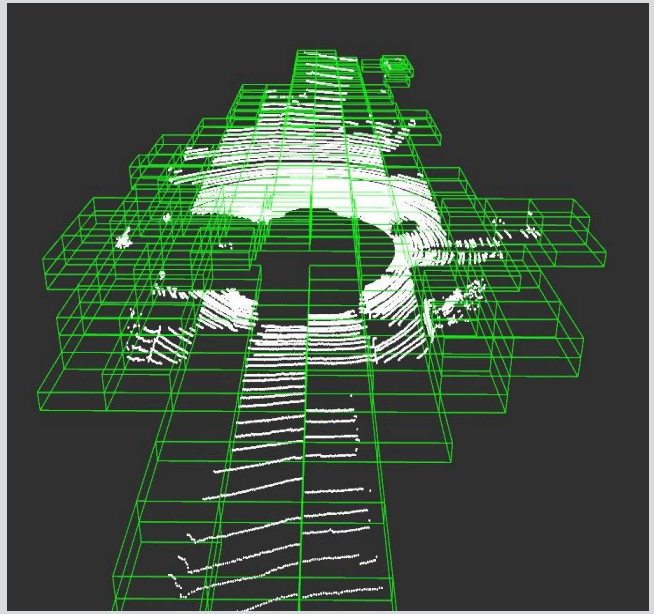
The segmentation operates through a **dual-phase grid representation**, ensuring stepwise refinement from coarse to precise results



Grid Representation

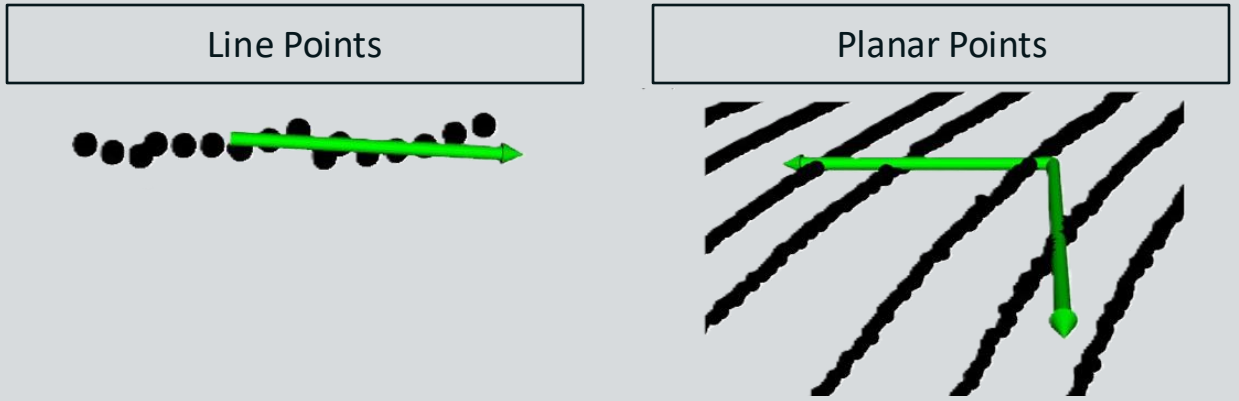
- Each point in the input point cloud is mapped to a cell in a regular 3D grid
- The grid serves as the foundation for local geometric analysis
- Each cell encapsulates surface properties e.g., slope, points, fitted plane, etc.
- The regular grid structure enables efficient neighborhood queries for segmentation and refinement





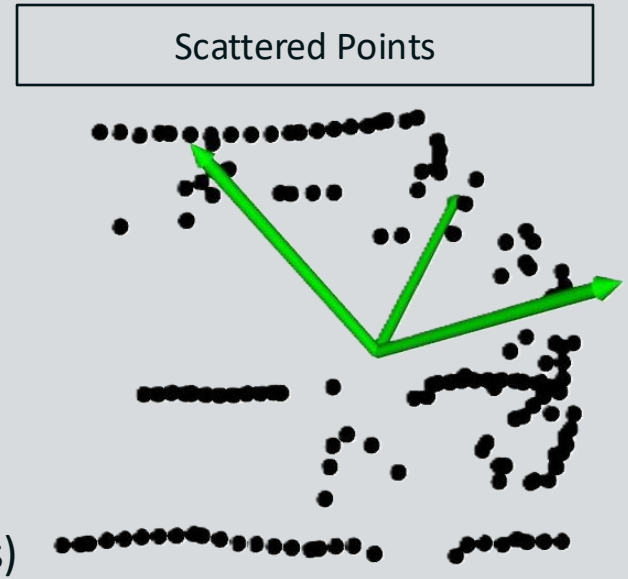
Aspect	Large Grid Cells	Small Grid Cells
Resolution	Low	High
Speed	Fast	Slow
Memory	Low usage	High usage
Detail	Coarse	Fine
Use Case	Initial processing, Tall structures	Fine segmentation, Complex Scenes

Eigen Classification



- **Extract Geometric Features**
  Covariance Matrix → Eigenvalues → Structural Type

- **Classify Local Structure**
  - Line: $\lambda_1 >> \lambda_2 \approx \lambda_3$ (1D dominance)
  - Planar: $\lambda_1 \approx \lambda_2 >> \lambda_3$ (2D dominance)
  - Scattered: $\lambda_1 \approx \lambda_2 \approx \lambda_3$ (3D distribution)



- **Identify Ground Surfaces**
  Tentative Ground = (Planar + Horizontal) or (Line + Horizontal)
  - Planar: Points form a flat surface (λ_3 is negligible)
  - Horizontal: Dominant direction is perpendicular to gravity

  Obstacles: All other cases (vertical planes, vertical lines, scattered points)

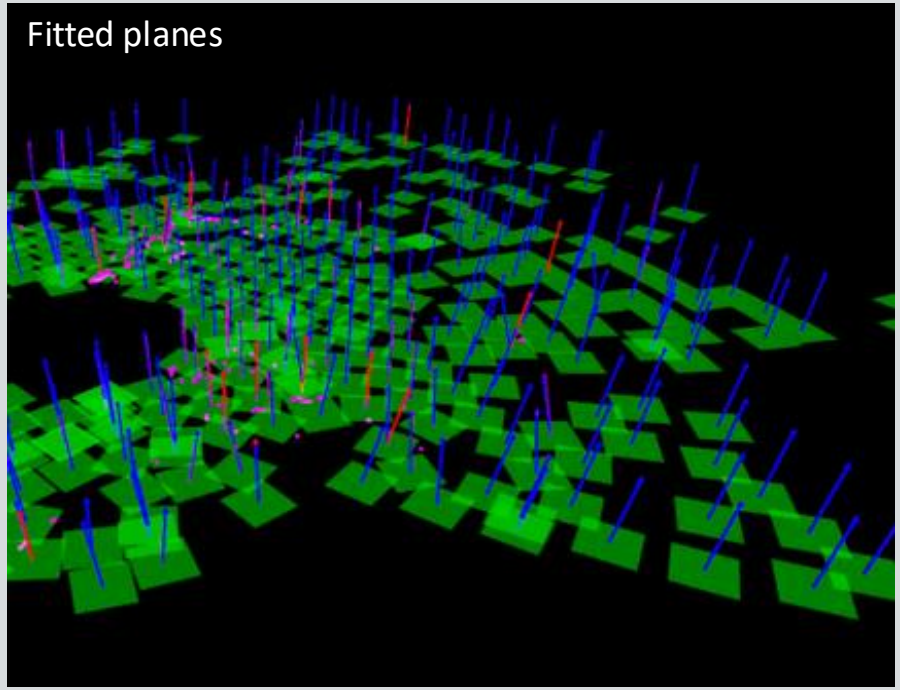
Surface Gradient Analysis

- **Plane Fitting with RANSAC**
  Input: 3D points in grid cell
  Output: Best-fit plane equation: $ax + by + cz + d = 0$

  RANSAC Advantages:
  - Robust to noise and outliers
  - Computationally efficient for this scale

- **Slope Calculation**
  Surface Normal: $n = (a, b, c)$
  Slope Angle: $\theta = \arccos(n \cdot z / |n|)$
  where $z = (0, 0, 1)$ is the gravity vector

- **Classification Rule**
  if $\theta < \theta_{thresh}$ → TENTATIVE GROUND (gently sloped surface)
  else → OBSTACLE (steep incline or vertical surface)



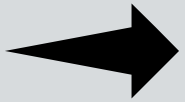
KD-Tree Ground Region Expansion



Challenges

Grid Dependency

- Traditional neighbor expansion fails when **cellsize < scan-line gaps**
- Causes **disconnected regions** and **poor recall**
- Forces a trade-off: **large cells (low precision)** vs. **small cells (poor connectivity)**





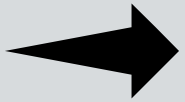
Solutions

KD-Tree Expansion

- Indexes all ground cell centroids in a **3D KD-Tree**
- Enables **radius-based neighbor search** beyond grid boundaries
- Allows **small cell sizes** without sacrificing **recall or connectivity**

Initial Seed Selection

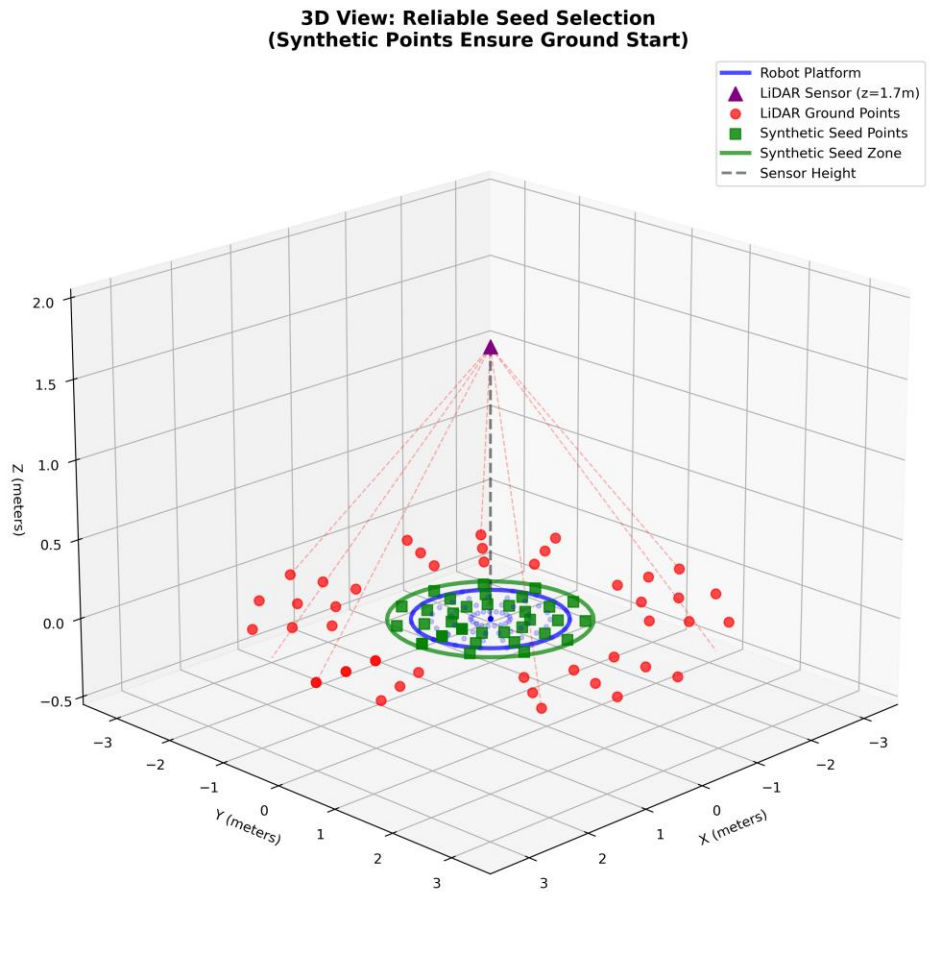
- Seed quality critical for expansion success
- Poor seed → under-segmentation/failure
- Sensor may not see ground below robot

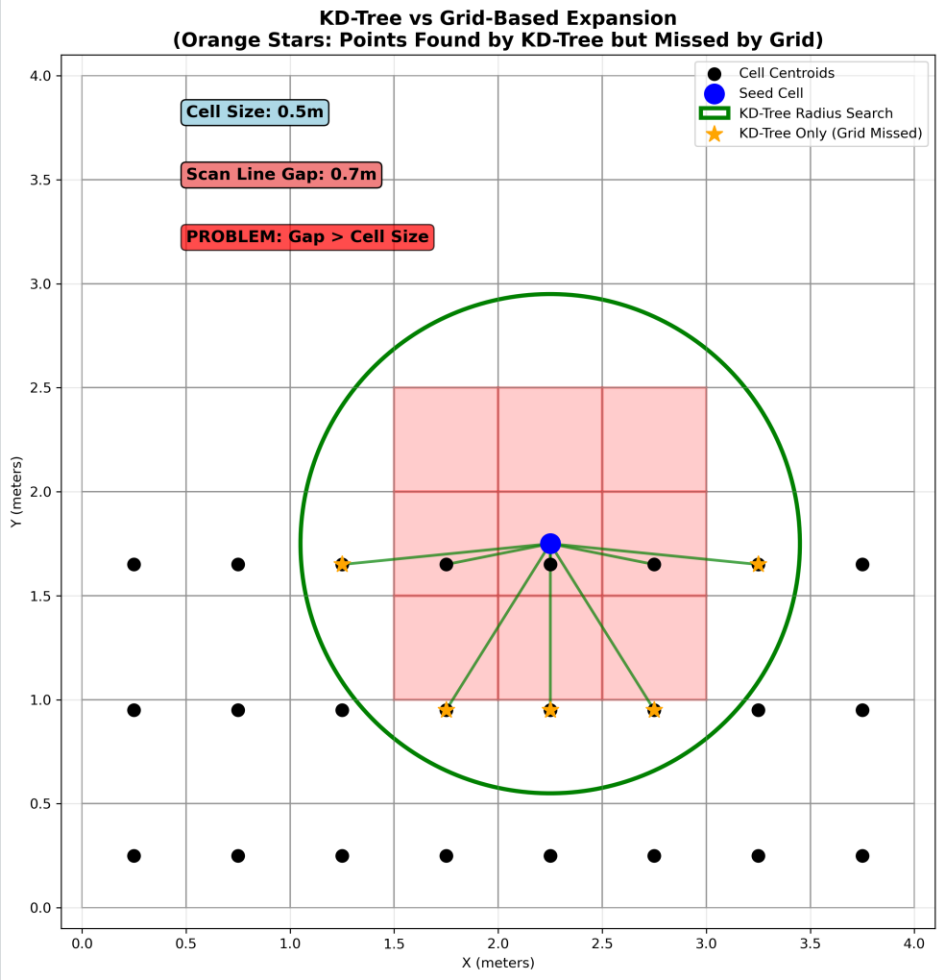


Reliable Seed Initialization

- Injects **synthetic points** directly below the robot
- Guarantees a **valid starting seed**, even with sensor gaps/occlusions
- Synthetic points are **removed before final output**

KD-Tree Ground Region Initialization and Expansion



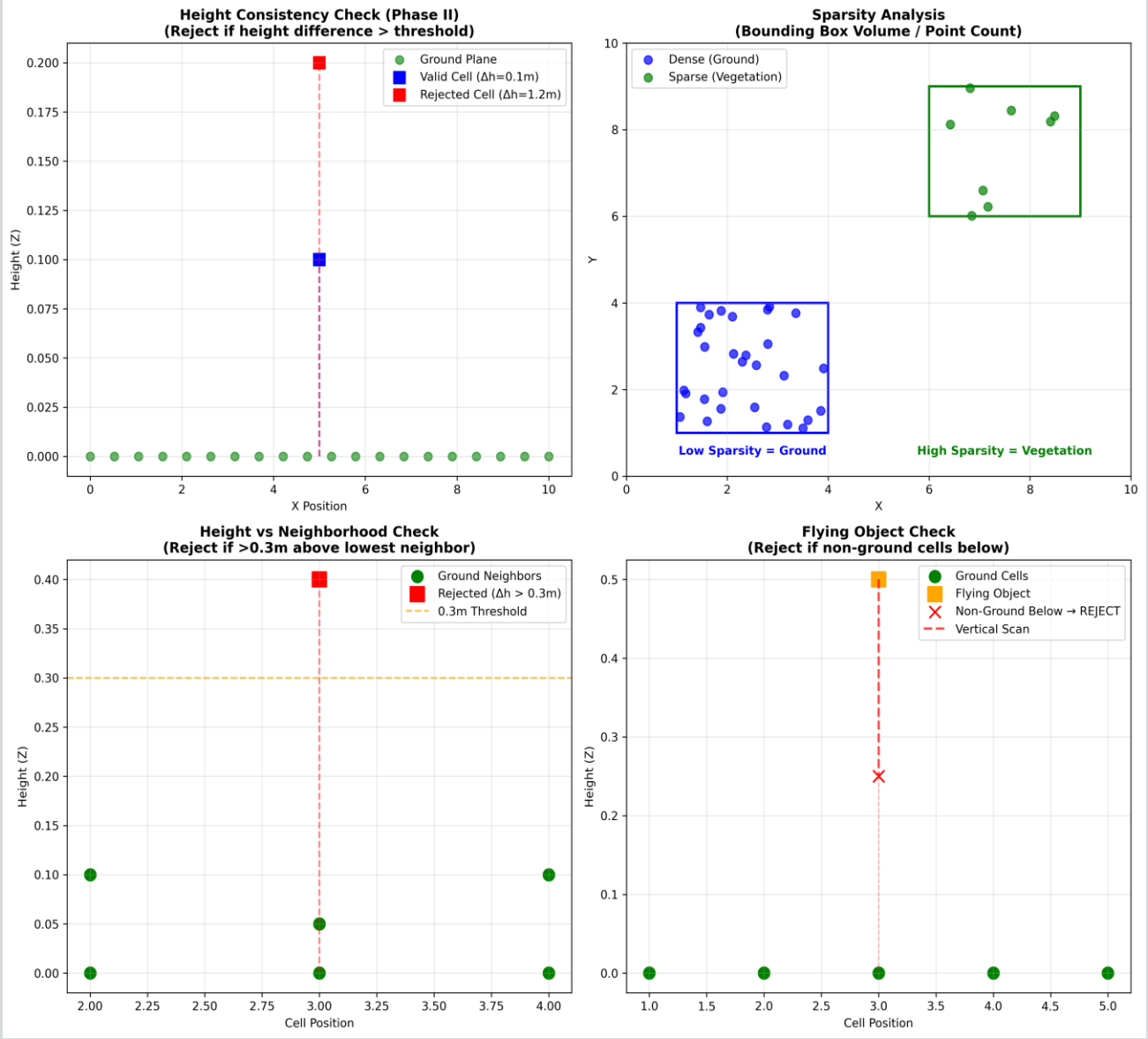


Ground Region Refinement

Height Consistency: In Phase II of ground expansion, reject neighbors with large height differences

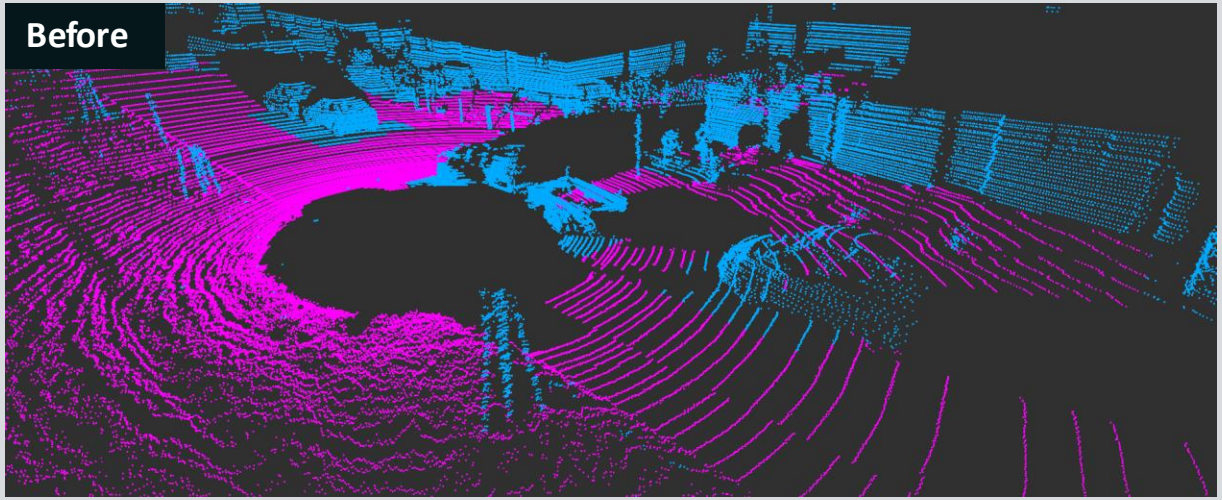
Each tentative ground cell after Phase II undergoes rigorous checks:

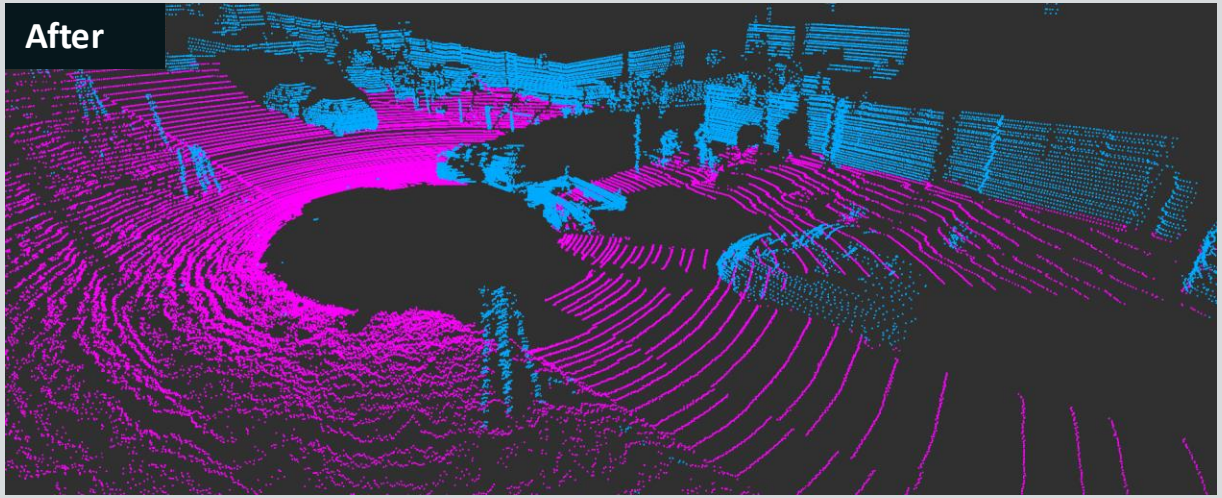
1. **Sparsity Ambiguity Check:** Compares 3D density of ground/non-ground inliers
2. **If Ambiguous** (scores are equal), triggers two extra validation steps:
   - **Height vs. Neighborhood:** Rejects cells elevated > 0.3m above the lowest local ground neighbor
   - **Flying Object Check:** Scans cells below; rejects if an occupied non-ground cell is found
3. **Final Classification:** Points are assigned to ground and non-ground points only after passing all validation stages



Under-Segmentation Correction

- **Targets Non-Ground Cells:** Processes cells initially classified as obstacles/non-ground
- **Finds Nearest Ground Reference:** Uses KD-tree to locate closest validated ground cell
- **Measures Height Relative to Ground:** Calculates point distances using ground plane normal vector
- **Reclassifies Low Points:** Points which are within threshold are reassigned as ground
- **Recovers Missing Ground:** Converts eligible non-ground cells back to ground classification





Evaluation Methodology

Experimental Setup

- **Baselines:** State-of-the-art ground segmentation algorithms
- **Dataset:** SemanticKITTI - urban LiDAR, Velodyne HDL-64E, 1.72m height
- **Metrics:** Precision, Recall, F1-score across 10-100m ranges
- **Reporting:** Mean ± std deviation across all distances

Precision = NTP / (NTP + NFP)
Recall = NTP/ (NTP + NFN)
F1-score = 2 · (Precision·Recall) / (Precision + Recall)

GSeg3D Configuration

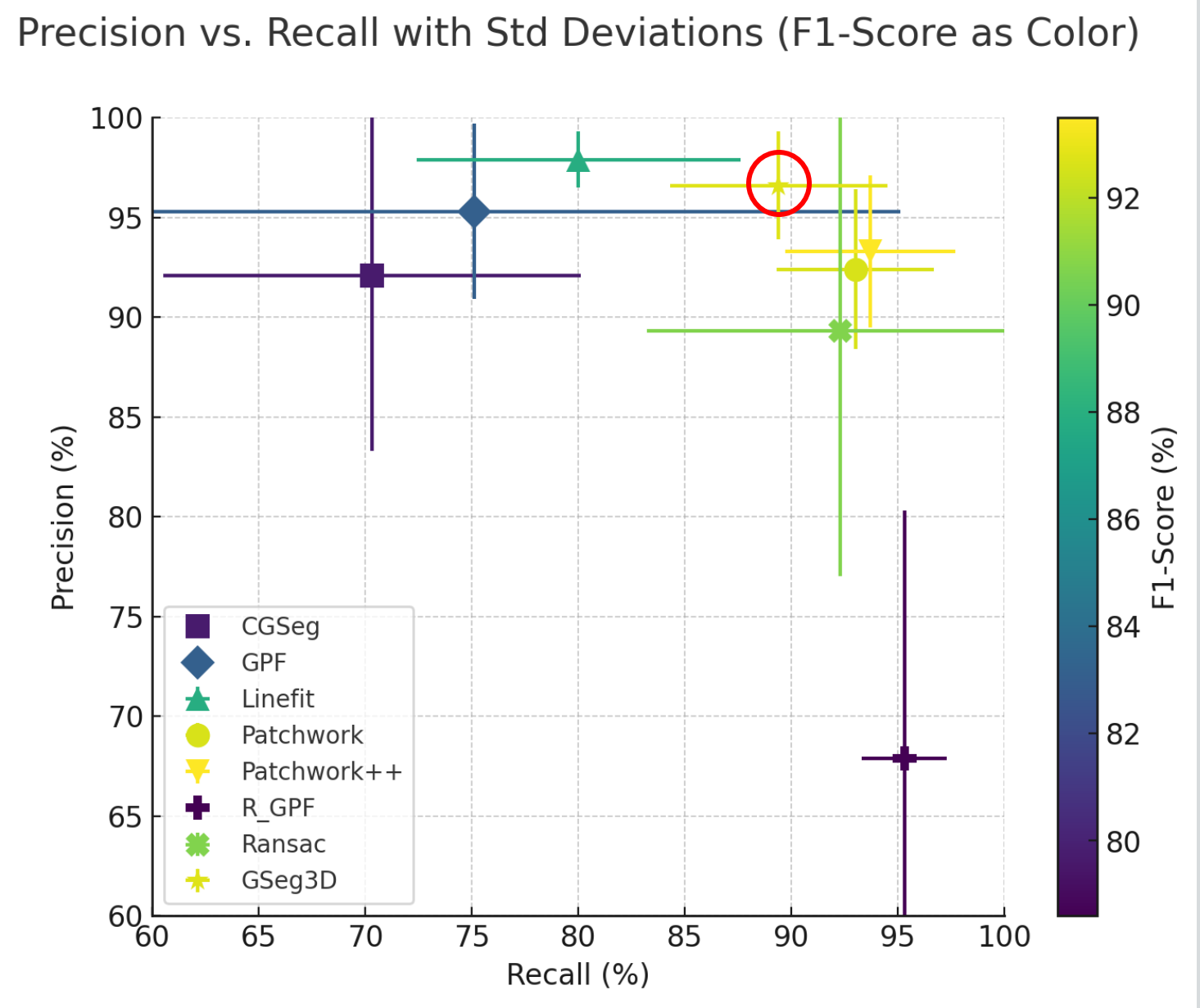
- **Grid Size:** (1.5, 1.0, 1.5)m (Phase I) & (1.5, 1.0, 0.2)m (Phase II)
- **Ground Inlier Threshold:** 0.125m
- **Ground Slope Threshold:** 30°
- **Robot Radius:** 2.7m

Experimental Platform

- Platform: Ubuntu 22.04 LTS
- Processor: Intel i7-11800H (8 cores, 16 threads)
- Memory: 62.5 GB RAM
- Evaluation: CPU-only implementation

Results Summary

- GSeg3D achieves the balance, delivering near-top precision (96.6%) while maintaining strong recall (89.4%).
- It avoids the extreme trade-offs of other methods, resulting in the best overall F1-score among high-precision algorithms.



Failure Cases & Future Directions



Vegetation Challenge & Semantic Fusion

- **Challenge:** Vegetation remains a primary source of error due to geometric ambiguity
- **Solution:** Integrating semantic segmentation to reliably distinguish ground from vegetation
- **Goal:** Improve ground-vegetation separation in complex environments



Runtime Optimization

- **Current Status:** 48 ms per scan provides competitive real-time performance
- **Focus:** Algorithmic refinements and parallelization for further speed improvements
- **Target:** Reduced latency for high-speed autonomous applications



Segmentation Refinement

- **Persistent Issue:** Under- and over-segmentation continue to affect recall metrics
- **Impact:** Limits ground point recovery in complex boundary cases
- **Path Forward:** Enhanced boundary refinement algorithms for fine-detail accuracy

Acknowledgments & Contact

- Acknowledgment: This work was funded by the German Federal Ministry of Education and Research (BMBF) under project Robdekon 2 (13N16537).

- Contact: haider_khan.lodhi@dfki.de
- GitHub: github.com/haider8645







References

- P. Pfreundschuh, H.F. Hendrikx, et al., "Dynamic Object Aware LiDAR SLAM based on Automatic Generation of Training Data," *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.